

TheKnife

Manuale Tecnico

Versione documento 1.0

Progetto di Laboratorio Interdisciplinare B — a.a. 2025/2026

Autori:

Ayoub Hammou — 761589
Esau Alessandro Argueta Zepeda — 761748

Sommario

1. Introduzione	4
2. Scelte Progettuali (UML).....	4
2.1 Architettura client/server	4
2.2 Protocollo applicativo.....	4
2.3 Diagramma UML delle classi (class diagram)	4
2.3.1 Package theknife.common — Modelli di dominio e protocollo.....	6
2.3.2 Package theknife.client — Livello di comunicazione.....	6
2.3.3 Package theknife.gui — Interfaccia utente	6
2.3.4 Package theknife.server — Livello server.....	7
2.4 Comportamento dinamico del sistema	7
2.4.1 Autenticazione (Login).....	8
.....	8
Descrizione del flusso	8
2.4.2 Ricerca dei ristoranti.....	9
Descrizione del flusso	9
2.4.3 Inserimento di un nuovo ristorante	10
.....	10
Descrizione del flusso	10
2.4.4 Inserimento di una recensione.....	11
.....	11
Descrizione del flusso	11
2.4.5 Risposta del gestore a una recensione.....	12
.....	12
Descrizione del flusso	12
3. Modello dei dati.....	13
3.1 Descrizione delle entità	14
Utenti.....	14
RistorantiTheKnife	14
Recensioni.....	15
Preferiti.....	15
4. Strutture dati utilizzate.....	16
5. Scelte algoritmiche	16
5.1 Ricerca geografica per raggio — formula di Haversine	16
5.2 Geocoding diretto e inverso	16
5.3 Validazione di coerenza tra fascia di prezzo e prezzo medio	17
5.4 Determinazione del simbolo di valuta dalla località	17
5.5 Calcolo di aggregazioni lato SQL.....	17
6. Analisi degli algoritmi	18

7. Pattern applicati	21
7.1 Pattern architetturali	21
7.1.1 Client-Server	21
7.1.2 Master/Worker (Master/Slave)	21
7.1.3 Architettura orientata ai messaggi (Message-oriented / Protocollo a operazioni)	21
7.2 Pattern di progettazione (GoF ed enterprise)	22
7.2.1 Singleton	22
7.2.2 Data Access Object (DAO)	22
7.2.4 Observer / Callback	22
7.2.5 Template Method	22
7.2.6 Factory (statica) / Utility	22
7.2.7 Data Transfer Object (DTO) / Value Object	22
7.2.8 Facade	23
Limiti della soluzione sviluppata	24
Scalabilità e prestazioni	24
Persistenza e affidabilità	24
Sicurezza	24
Qualità e completezza dei dati	24
Internazionalizzazione e interfaccia	24
Sitografia / Bibliografia	25

1. Introduzione

TheKnife è un'applicazione client-server per la ricerca, la consultazione e la recensione di ristoranti. TheKnife adotta un'architettura client-server a due livelli applicativi con un livello di persistenza separato (three-tier logico): un client desktop Swing comunica con un server multi-thread tramite un protocollo applicativo proprietario veicolato su socket TCP; il server, a sua volta, si appoggia a un database relazionale PostgreSQL tramite JDBC. Il package `theknife.common` è condiviso tramite dipendenza di compilazione sia dal client sia dal server, e costituisce il contratto (protocollo + DTO) tra i due sottosistemi.

2. Scelte Progettuali (UML)

2.1 Architettura client/server

Il sistema è strutturato in quattro processi distinti che comunicano attraverso una connessione socket:

- Client (AppGUI): applicazione desktop Swing che espone le funzionalità agli utenti finali (login, registrazione, ricerca, gestione preferiti, recensioni, gestione ristoranti). Ogni interazione con il server avviene tramite la classe `ClientTK`, che incapsula il socket e implementa un pattern richiesta/risposta sincrono e thread-safe (metodo `invia()`, dichiarato `synchronized`).
- Server (ServerTK): applicazione con interfaccia di controllo (ServerGUI) che permette di configurare la connessione al database, avviare/fermare il listener TCP e monitorare in tempo reale le operazioni tramite un pannello di log. Per ogni client connesso viene istanziato un `ClientHandler`, eseguito su un thread separato (implementa `Runnable`), che legge le richieste e produce le risposte.
- Database (PostgreSQL): la persistenza è interamente delegata a `DatabaseManager`, unico punto di accesso alle tabelle utenti, `RistorantiTheKnife`, recensioni e preferiti.
- Servizio esterno (Nominatim): utilizzato sia dal client (per convertire una città inserita dall'utente in coordinate geografiche prima di inviare una ricerca) sia come fallback lato interfaccia quando l'utente non specifica manualmente latitudine e longitudine in fase di inserimento di un nuovo ristorante.

2.2 Protocollo applicativo

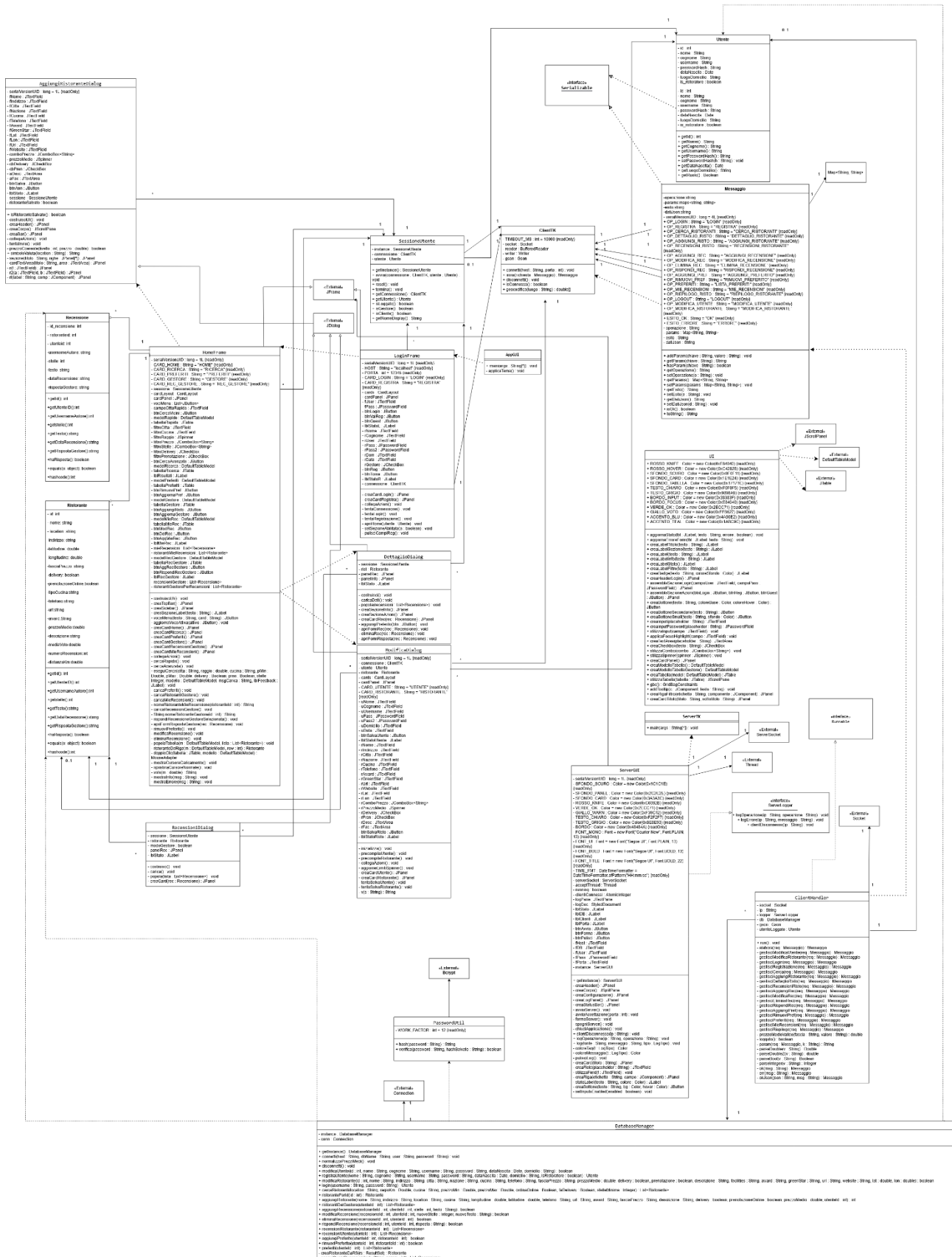
La comunicazione fra client e server è incapsulata in un unico oggetto di trasporto, la classe `Messaggio` (package `theknife.common`), serializzata in JSON e scambiata come singola riga di testo su un `BufferedReader/Writer` costruito sopra il socket. Un messaggio di richiesta contiene un codice operazione (campo `operazione`, valorizzato con una delle costanti `OP_LOGIN`, `OP_REGISTRA`, `OP_CERCA_RISTORANTI`, `OP_AGGIUNGI_RISTO`, `OP_AGGIUNGI_REC`, `OP_RISPONDI_REC`, ecc.) e una mappa di parametri testuali (`params`). Il messaggio di risposta riporta un esito (`ESITO_OK` / `ESITO_ERRORE`), un messaggio testuale opzionale (in caso di errore) e, se previsto, un payload dati serializzato come stringa JSON (campo `datiJson`), che il client de-serializza nell'oggetto di dominio atteso (`Utente`, `Ristorante`, `Recensione` o liste di questi).

Il dispatching lato server avviene nel metodo `ClientHandler.elabora()`, che seleziona l'handler applicativo corretto in base al campo `operazione` tramite uno switch, delegando la logica di business e l'accesso ai dati a `DatabaseManager`.

2.3 Diagramma UML delle classi (class diagram)

Il diagramma seguente rappresenta la struttura statica del sistema ad un livello di astrazione superiore rispetto al codice sorgente: sono mostrate le classi principali dei quattro package, i rispettivi attributi/operazioni essenziali e le relazioni di associazione, dipendenza e utilizzo fra di esse (incluse le classi

esterne delle librerie Swing, Gson/BCrypt e l'interfaccia Serializable implementata dai modelli di dominio, condizione necessaria al loro trasporto via rete).



2.3.1 Package theknife.common — Modelli di dominio e protocollo

Questo package non dipende da nessun altro package del progetto ed è condiviso sia dal client sia dal server: tutte le sue classi implementano `Serializable` perché, sebbene la serializzazione effettiva avvenga in JSON tramite Gson, la marcatura è mantenuta come contratto di trasportabilità sulla rete:

- **Messaggio** — contenitore di comunicazione. Espone operazione (codice della richiesta), una `Map<String,String>` params, l'esito (OK/ERRORE) e un payload `datiJson` opzionale. Le costanti statiche `OP_*` definiscono l'intero vocabolario delle operazioni supportate dal protocollo (login, registrazione, ricerca, CRUD ristoranti/recensioni/preferiti, ecc.).
- **Utente** — rappresenta un cliente o un gestore. Contiene i dati anagrafici, lo username, l'hash della password (mai inviato dopo il login) e il flag `is_ristoratore` che discrimina il ruolo.
- **Ristorante** — rappresenta un locale: dati identificativi e di localizzazione (incluse coordinate geografiche), fascia di prezzo, servizi offerti (delivery, prenotazione online), oltre a campi calcolati lato server (`mediaVoto`, `numeroRecensioni`, `distanzaKm`) derivati dalle query di ricerca.
- **Recensione** — rappresenta la recensione di un cliente su un ristorante: voto in stelle, testo, data e un'eventuale `rispostaGestore` (al più una per recensione).

2.3.2 Package theknife.client — Livello di comunicazione

- **ClientTK** — incapsula il Socket verso il server e i relativi stream di lettura/scrittura. Il metodo `invia()` implementa il pattern sincrono richiesta/risposta, con gestione dei timeout (`SocketTimeoutException`) e degli errori di protocollo. La stessa classe ospita, come metodo statico, la funzionalità di geocodifica (`geocodifica(String luogo)`), che interroga il servizio Nominatim via `URLConnection` e restituisce una coppia di coordinate.

2.3.3 Package theknife.gui — Interfaccia utente

- **AppGUI** — entry point dell'applicazione client: imposta il tema grafico scuro globale e avvia `LoginFrame` sull'Event Dispatch Thread.
- **LoginFrame** — gestisce, tramite un `CardLayout` interno, le schermate di login e registrazione; stabilisce la connessione iniziale al server tramite `ClientTK` e, in caso di successo, avvia la sessione applicativa.
- **SessioneUtente** — singleton che mantiene lo stato globale della sessione client: la connessione `ClientTK` attiva e l'utente autenticato (null in modalità ospite). Espone metodi di comodo per interrogare il ruolo corrente (`isLoggato`, `isCliente`, `isGestore`).
- **HomeFrame** — schermata principale post-login, organizzata anch'essa con `CardLayout` in più sezioni (Home, Ricerca, Preferiti, I miei ristoranti, Recensioni ricevute, Le mie recensioni), visibili in base al ruolo dell'utente.
- **DettaglioDialog / RecensioniDialog** — finestre di dettaglio di un ristorante: mostrano informazioni, elenco recensioni e, in base al ruolo, azioni disponibili (aggiungi ai preferiti, scrivi/modifica/elimina recensione, rispondi come gestore).
- **AggiungiRistoranteDialog / ModificaDialog** — form per l'inserimento di un nuovo ristorante e per la modifica del profilo utente o di un ristorante esistente; effettuano validazioni lato client e, se necessario, richiedono la geocodifica automatica dell'indirizzo.
- **UI** — classe di utilità statica che centralizza la palette colori e le factory dei componenti Swing stilizzati (bottoni, campi di testo, tabelle, card), garantendo coerenza visiva fra tutte le schermate.

2.3.4 Package theknife.server — Livello server

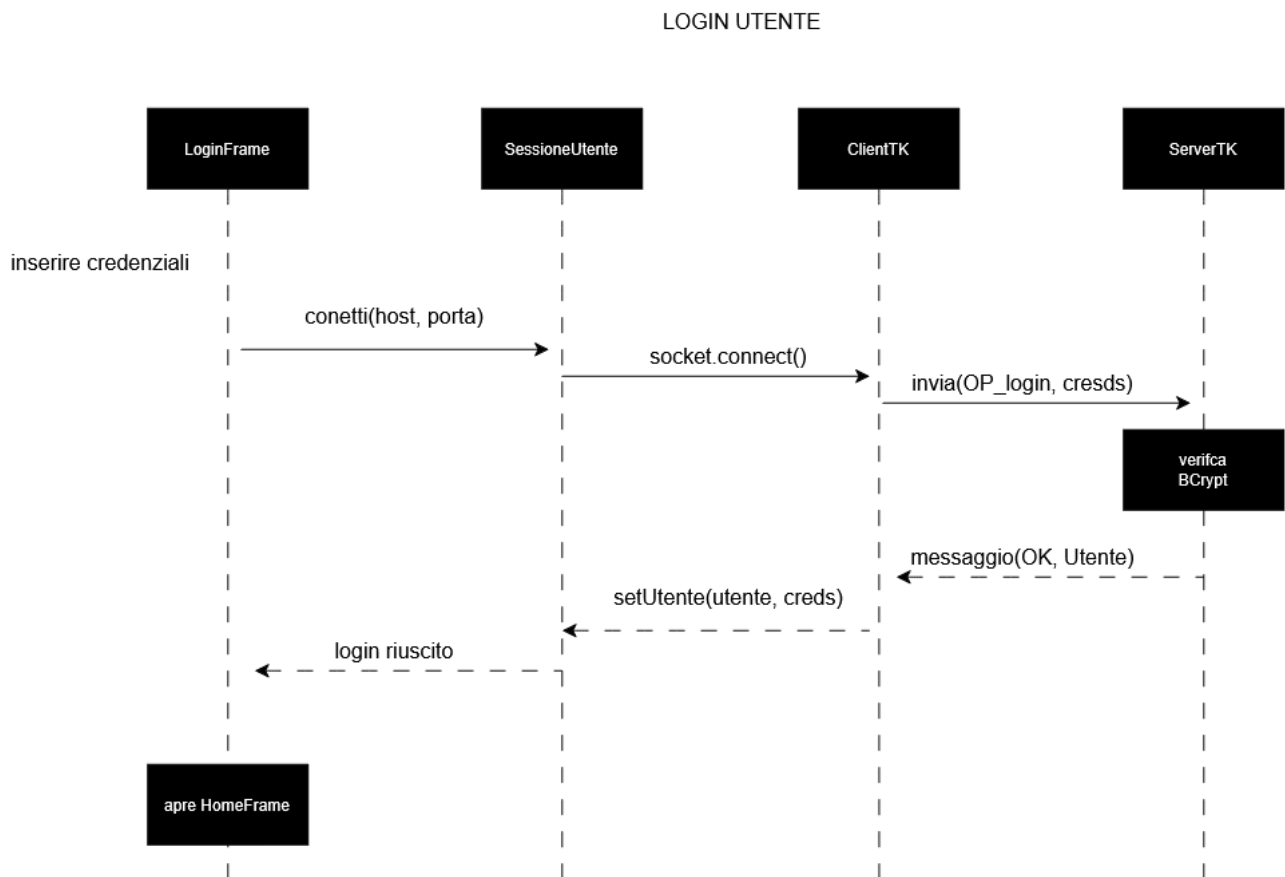
- **ServerTK** — entry point del server: avvia ServerGUI sull'Event Dispatch Thread.
- **ServerGUI** — pannello di controllo grafico del server: configurazione della connessione al database, avvio/arresto del ServerSocket, log delle operazioni in tempo reale e conteggio dei client connessi.
- **ClientHandler** — implementa Runnable; gestisce il ciclo di vita di una singola connessione client, effettua il parsing dei messaggi JSON in ingresso e ne gestisce l'elaborazione tramite il metodo elabora(), che effettua il dispatch sulla base del campo operazione.
- **DatabaseManager** — centralizza tutte le interazioni con PostgreSQL tramite JDBC/PreparedStatement: autenticazione e registrazione utenti, ricerca geolocalizzata dei ristoranti (formula di Haversine calcolata in SQL), CRUD di ristoranti, recensioni e preferiti.
- **PasswordUtil** — utilità stateless per l'hashing (BCrypt.withDefaults().hashToString) e la verifica a tempo costante delle password (BCrypt.verifyer().verify), utilizzata da DatabaseManager in fase di registrazione e login.

2.4 Comportamento dinamico del sistema

La struttura statica descritta nel capitolo precedente viene qui completata dalla vista dinamica del sistema. I diagrammi di sequenza seguenti descrivono, per i principali casi d'uso, l'ordine delle interazioni fra gli oggetti coinvolti (interfaccia grafica, livello di comunicazione client, server e livello di accesso ai dati), evidenziando in particolare i controlli di autorizzazione applicati lato server prima di ogni operazione di scrittura sul database (blocchi alt/else).

Ogni caso d'uso condivide lo stesso pattern architetturale di fondo: la GUI costruisce un oggetto Messaggio con l'operazione e i parametri necessari; ClientTK.invia() lo trasmette in modo sincrono al server; ClientHandler.elabora() lo instrada verso l'handler applicativo corretto, che verifica le precondizioni (autenticazione, ruolo, proprietà della risorsa) e, se soddisfatte, invoca il metodo corrispondente di DatabaseManager; la risposta (esito ed eventuale payload) risale la stessa catena fino ad aggiornare l'interfaccia.

2.4.1 Autenticazione (Login)

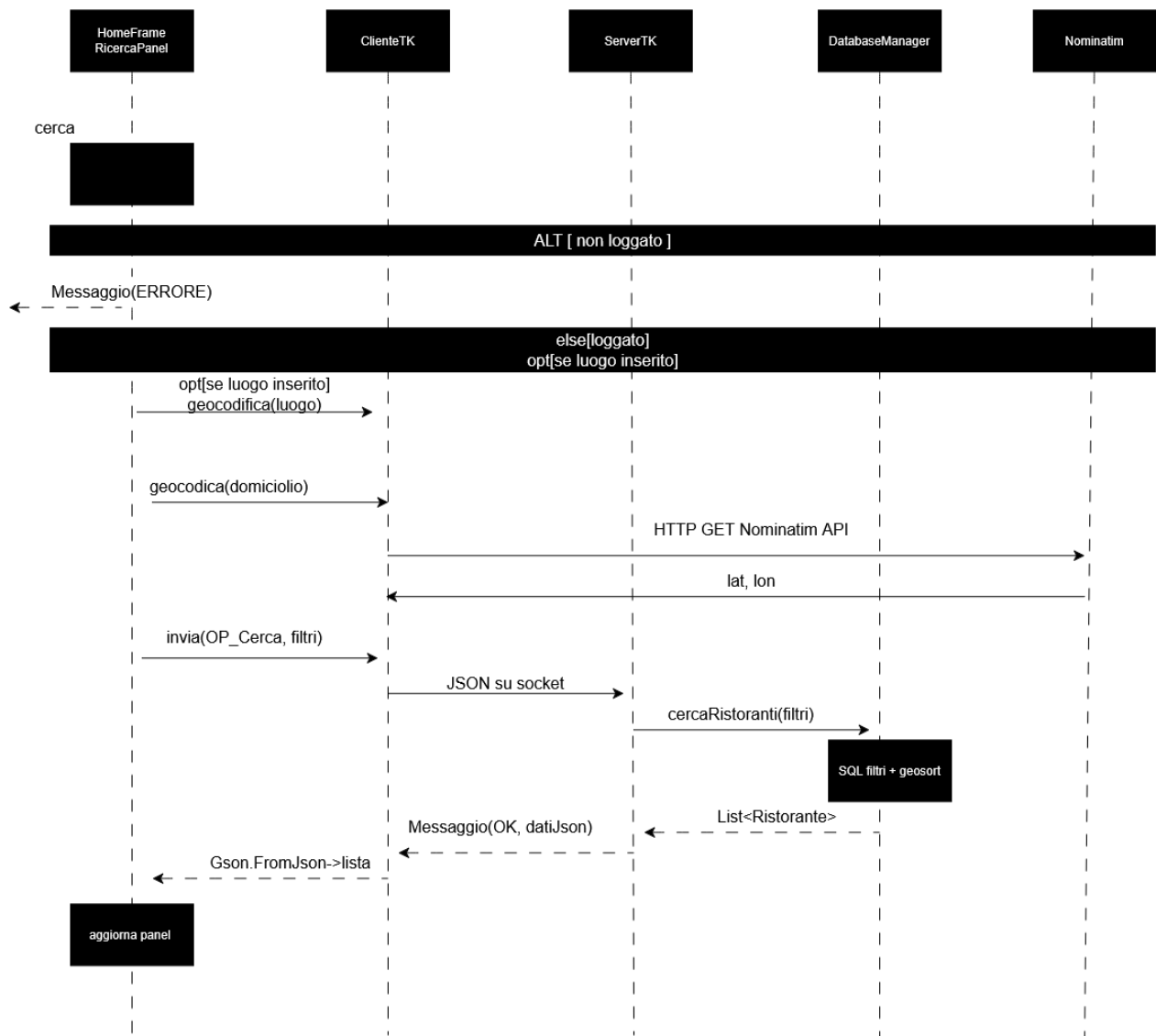


Descrizione del flusso

- **LoginFrame.tentaLogin()** — raccoglie username e password dal form e costruisce un Messaggio con operazione OP_LOGIN.
- **ClientTK.invia()** — inoltra sincronicamente la richiesta al server sulla connessione già stabilita da LoginFrame.tentaConnessione().
- **ServerTK** — opera in background attraverso i due seguenti metodi:
 - **ClientHandler.elabora() → gestisciLogin()** — sul server, recupera l'utente tramite DatabaseManager.login(username, password).
 - **DatabaseManager.login()** — esegue una SELECT sulla tabella utenti per username e, se trovato, verifica la password in chiaro fornita contro l'hash salvato tramite PasswordUtil.verifica() (confronto BCrypt a tempo costante).
- **Esito** — se le credenziali sono corrette il server imposta utenteLoggato sulla connessione e restituisce un Messaggio con esito OK e l'oggetto Utente serializzato in datiJson; in caso contrario restituisce un errore ("Username o password errati").
- **Lato client** — in caso di successo, SessionUtente.avvia() memorizza connessione e utente, LoginFrame apre HomeFrame e si chiude (dispose).

2.4.2 Ricerca dei ristoranti

Ricerca ristoranti

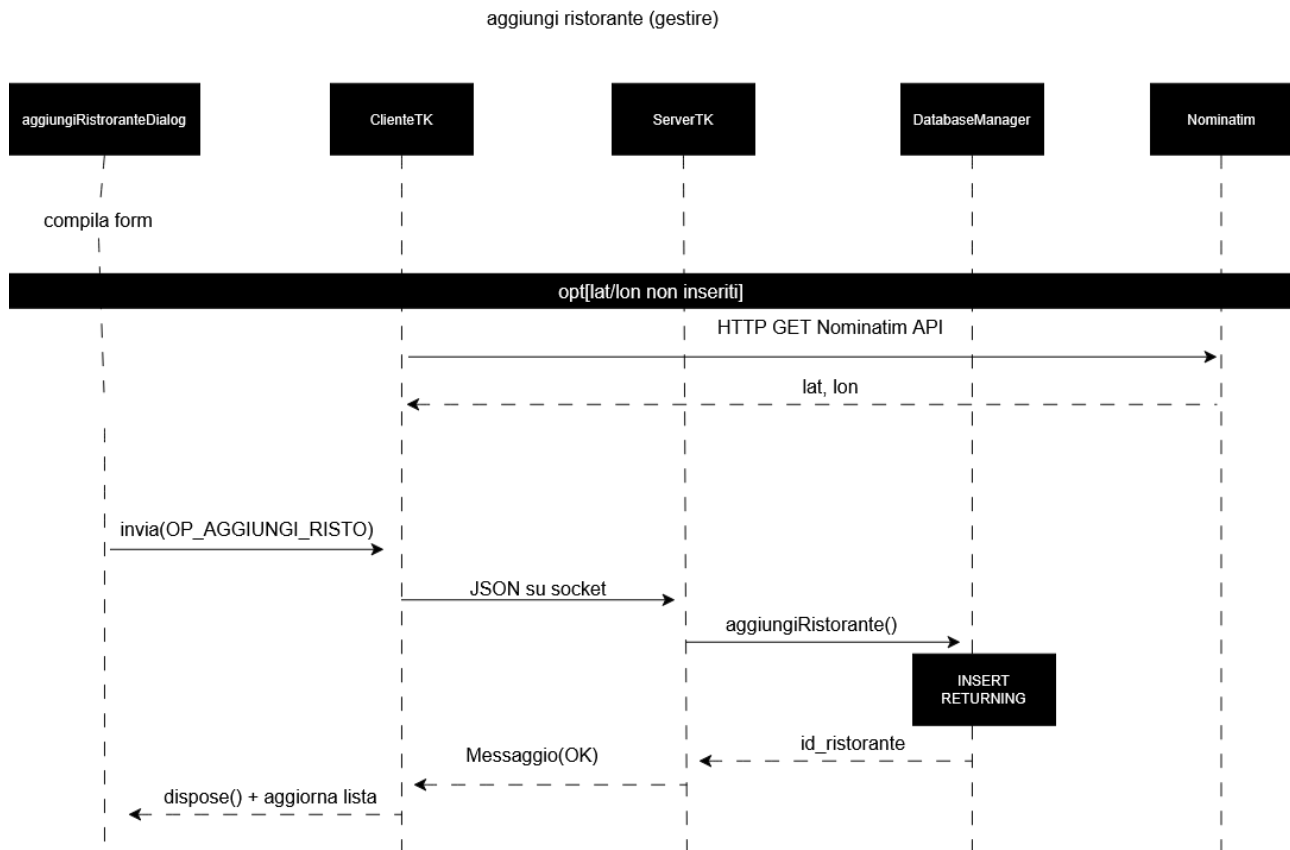


Descrizione del flusso

- **Geocodifica lato client** — prima di interrogare il server, HomeFrame (nei metodi `cercaRapida()`/`eseguiCerca()`) converte la città digitata in coordinate tramite `ClientTK.geocodifica()`, che invoca il servizio esterno Nominatim.
- **Regole di business sull'input** — se l'utente non è autenticato (ospite), il server richiede obbligatoriamente una città; se è autenticato ma non fornisce alcun input, il server utilizza automaticamente il luogo di domicilio salvato nel suo profilo.
- **Messaggio inviato** — un Messaggio con operazione `OP_CERCA_RISTORANTI`, contenente latitudine/longitudine (o città), raggio di ricerca e i filtri opzionali (cucina, fascia di prezzo, delivery, prenotazione online, stelle minime).
- **DatabaseManager.cercaRistoranti()** — compone dinamicamente una query SQL che calcola la distanza reale fra il punto di ricerca e ciascun ristorante tramite la formula di Haversine (espressa con le funzioni trigonometriche di PostgreSQL), applica i filtri richiesti e ordina i risultati per distanza crescente e voto medio decrescente.

- **Risposta** — la lista di oggetti Ristorante (con `mediaVoto`, `numeroRecensioni` e `distanzaKm` calcolati) viene serializzata e restituita al client, che popola la tabella dei risultati; un doppio click su una riga apre `DettaglioDialog`.

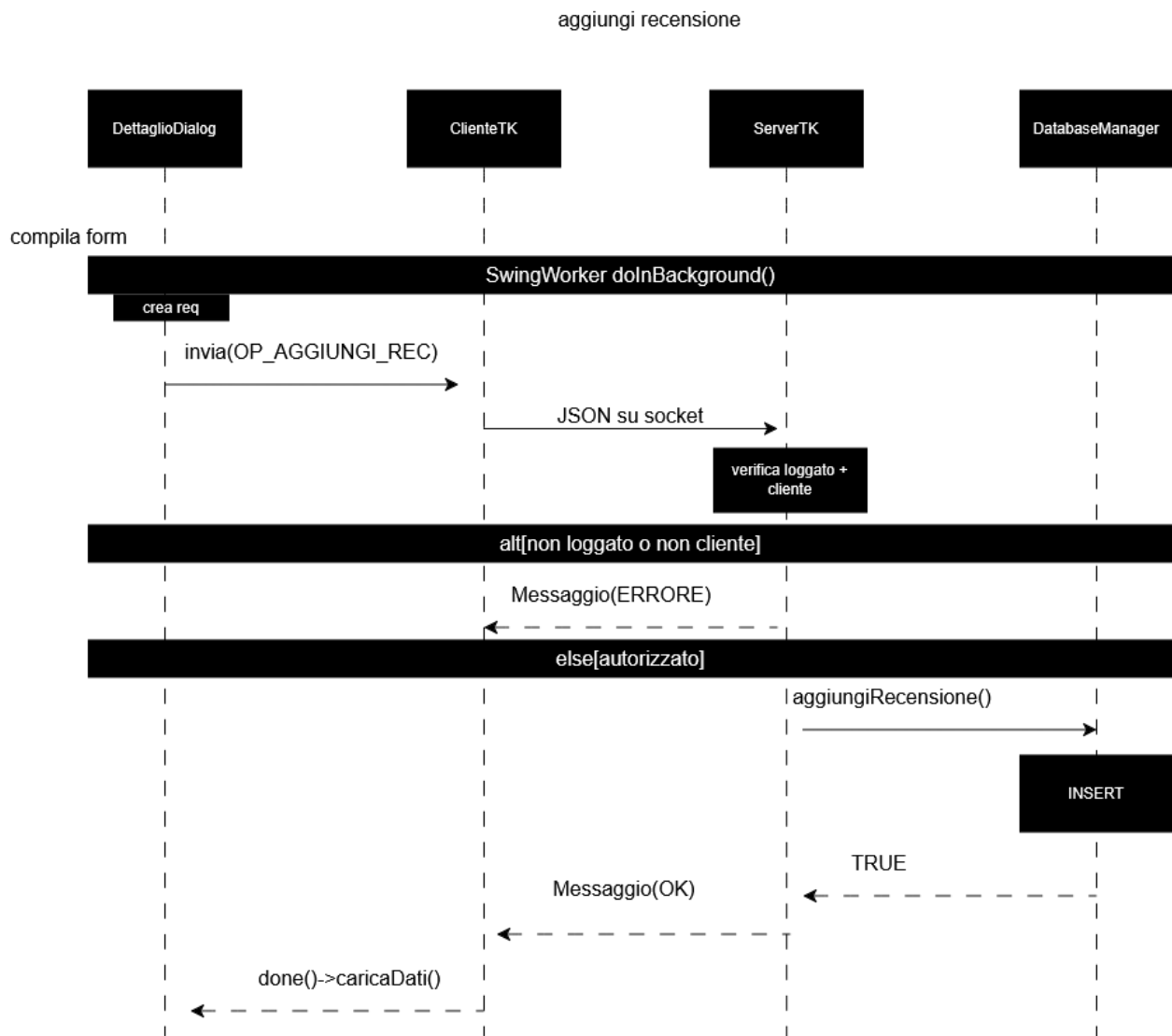
2.4.3 Inserimento di un nuovo ristorante



Descrizione del flusso

- **Validazione lato client** — `AggiungiRistoranteDialog.tentaInvio()` verifica la presenza dei campi obbligatori e la coerenza fra fascia di prezzo selezionata e prezzo medio indicato (`prezzoCoerente()`).
- **Geocodifica opzionale** — se l'utente non inserisce manualmente latitudine e longitudine, il client invoca `ClientTK.geocodifica()` sul servizio Nominatim per ricavarle automaticamente a partire da città e nazione (ramo `opt[lat/lon non inseriti]` del diagramma).
- **Messaggio inviato** — un `Messaggio` con operazione `OP_AGGIUNGI_RISTO`, contenente tutti gli attributi del nuovo ristorante.
- **Autorizzazione lato server** — `gestisciAggiungiRistorante()` verifica che l'utente sia autenticato e che abbia ruolo di gestore, prima di procedere.
- **DatabaseManager.aggiungiRistorante()** — esegue una `INSERT ... RETURNING id_ristorante` sulla tabella `RistorantiTheKnife`, associando il nuovo record all'utente gestore che lo ha creato (campo `utente_id`).
- **Risposta** — il server restituisce l'esito dell'operazione con l'identificativo generato; il dialog mostra conferma e si chiude automaticamente.

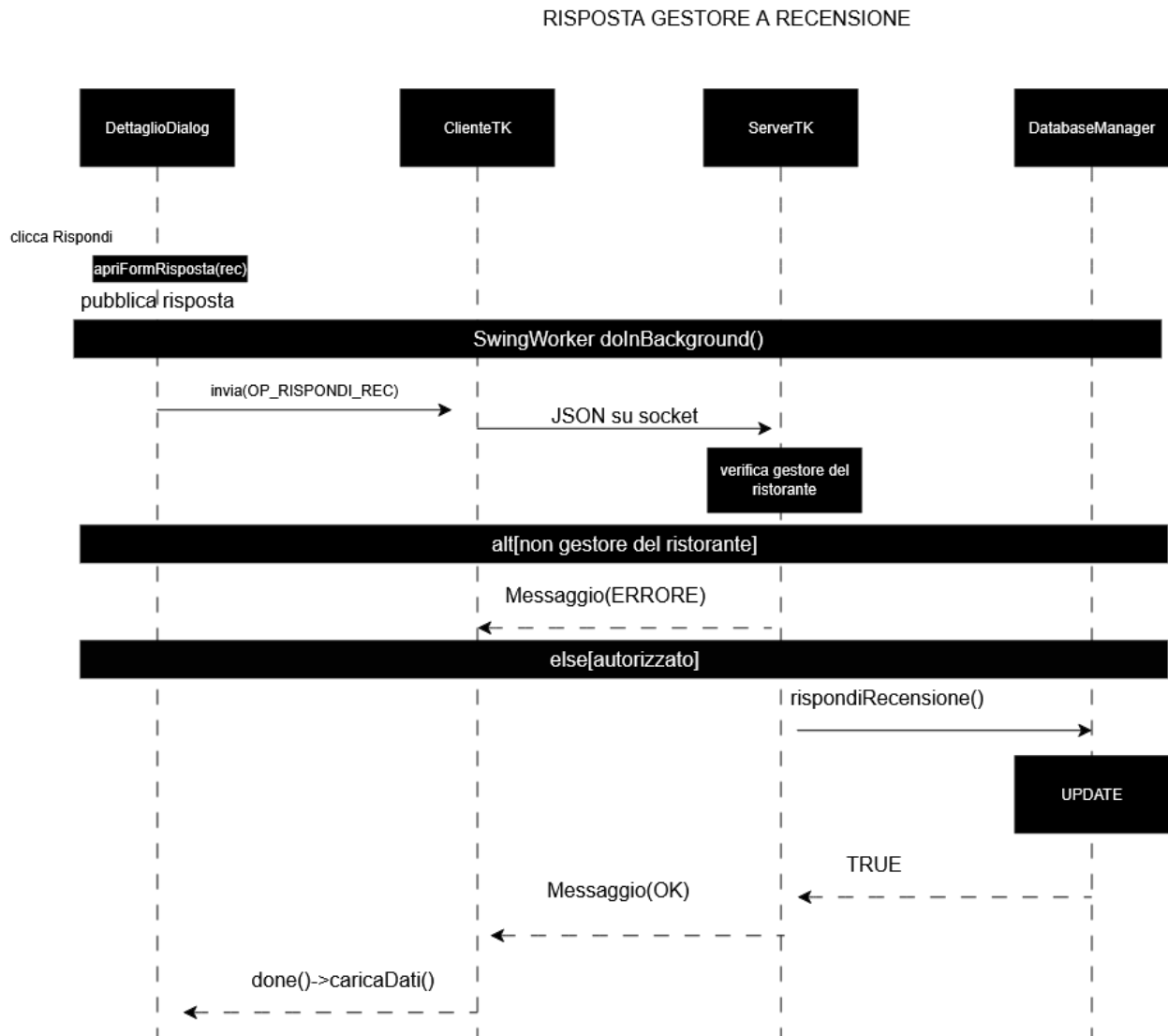
2.4.4 Inserimento di una recensione



Descrizione del flusso

- **Costruzione della richiesta** — `DettaglioDialog.apriFormRec()` raccoglie voto (1-5 stelle) e testo, quindi costruisce un `Messaggio` con operazione `OP_AGGIUNGI_REC` (o `OP_MODIFICA_REC` in caso di modifica di una recensione esistente), eseguito in background tramite `SwingWorker` per non bloccare l'interfaccia.
- **Controllo di autorizzazione** — sul server, `gestisciAggiungiRec()` verifica che l'utente sia autenticato e che il suo ruolo sia `Cliente` (i gestori non possono recensire): è il blocco `alt[non loggato o non cliente] / else[autorizzato]` del diagramma.
- **DatabaseManager.aggiungiRecensione()** — se autorizzato, esegue una `INSERT` sulla tabella recensioni associando la recensione al ristorante e all'utente.
- **Aggiornamento della vista** — in caso di esito positivo il client ricarica l'elenco delle recensioni del ristorante (`caricaDati()`), mostrando la nuova recensione nell'interfaccia.

2.4.5 Risposta del gestore a una recensione

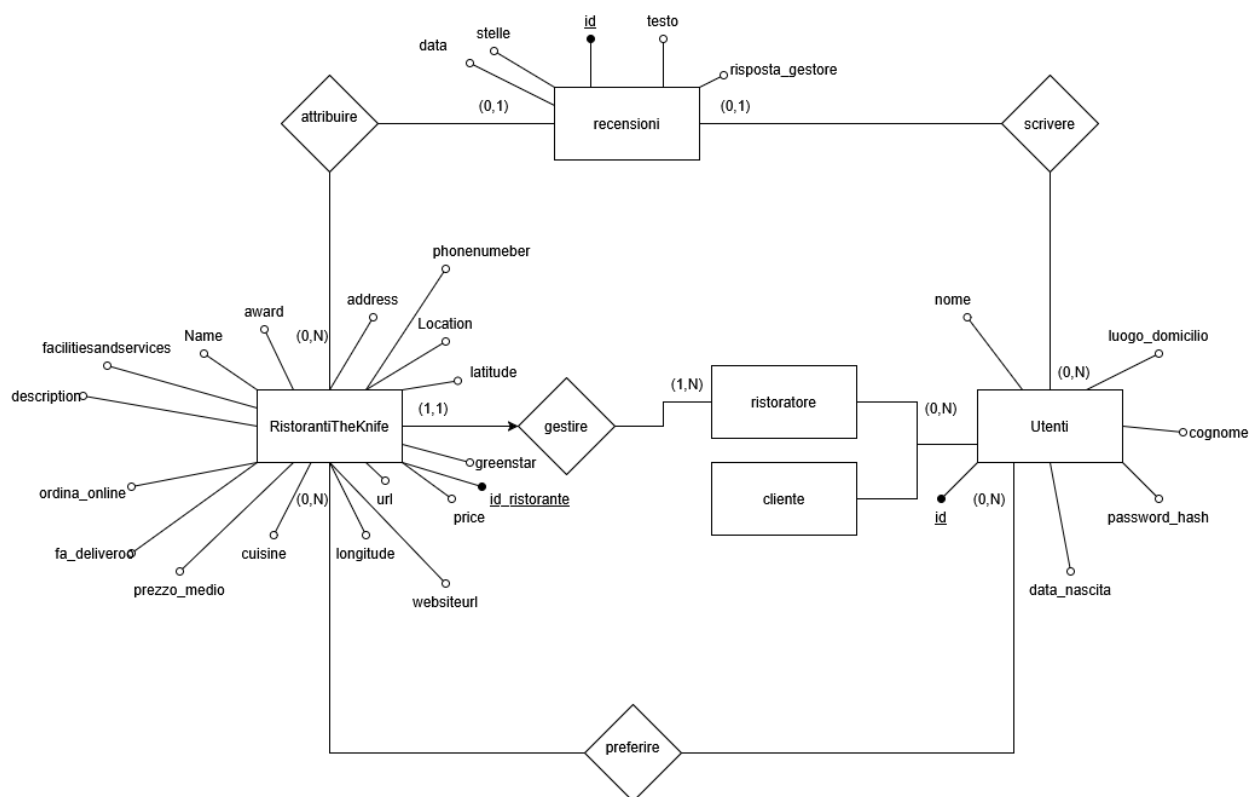


Descrizione del flusso

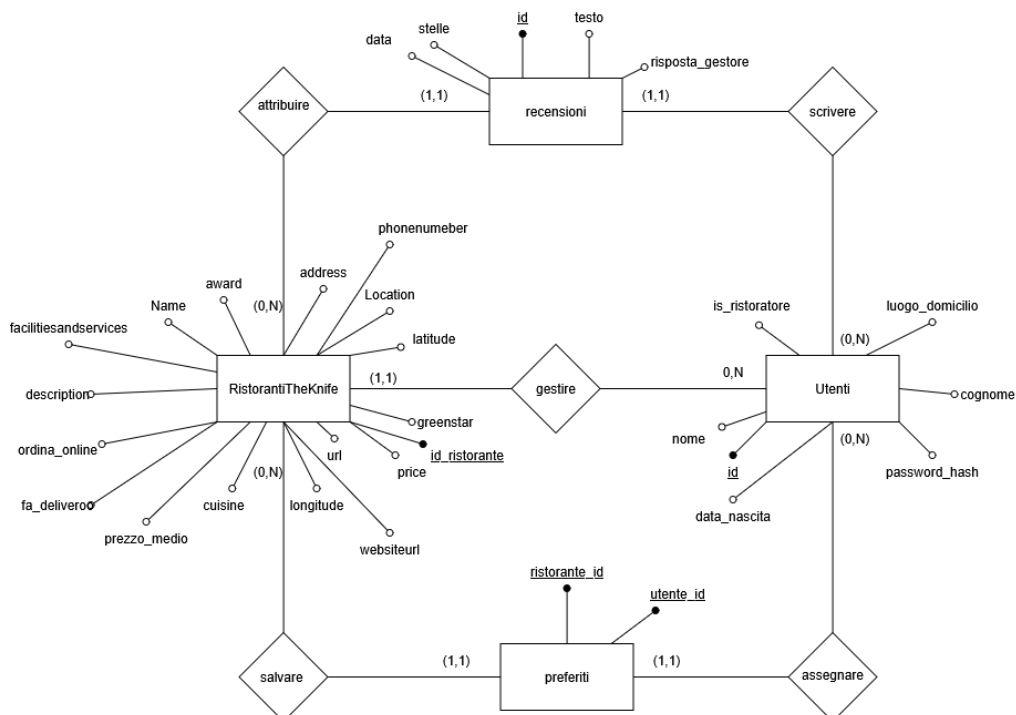
- **Apertura del form** — dalla schermata "Recensioni ricevute" (o dal dettaglio del ristorante), il gestore seleziona una recensione e apre il form di risposta tramite `apriFormRisposta(rec)` / `apriFormRispostaGestore(rec)`.
- **Messaggio inviato** — un `Messaggio` con operazione `OP_RISPONDI_REC`, contenente l'identificativo della recensione e il testo della risposta, inviato tramite un `SwingWorker` in background.
- **Controllo di proprietà** — sul server, `gestisciRispondiRec()` verifica non solo che l'utente sia un gestore, ma che il ristorante a cui appartiene la recensione sia effettivamente di sua proprietà: è il blocco `alt[non gestore del ristorante] / else[autorizzato]` del diagramma, implementato tramite una JOIN fra recensioni e `RistorantiTheKnife` filtrata su `utente_id`.
- **DatabaseManager.rispondiRecensione()** — se autorizzato, esegue una `UPDATE` sulla tabella recensioni valorizzando il campo `risposta_gestore` (operazione idempotente: consente anche la modifica di una risposta già data).
- **Aggiornamento della vista** — il client ricarica le recensioni e mostra la risposta pubblicata sotto la recensione corrispondente.

3. Modello dei dati

La persistenza è affidata a un database relazionale PostgreSQL, interrogato dal server tramite JDBC con Statement parametrizzati. Il diagramma E/R iniziale non ristrutturato è il seguente:



Il seguente diagramma E/R ristrutturato, è composto da quattro entità principali.



3.1 Descrizione delle entità

Utenti

Rappresenta sia i clienti sia i gestori di ristoranti, distinti dal flag booleano `is_ristoratore` (pattern «single table per ruolo»). Lo username è vincolato UNIQUE a livello di schema; la violazione del vincolo (SQLState che inizia per «23») viene intercettata in `DatabaseManager.registraUtente` per restituire un esito applicativo di «username già in uso» senza una query di controllo preventiva (evitando così una race condition check-then-act).

Qui le query SQL usate per la creazione della tabella nel database:

```
CREATE TABLE utenti (
  id SERIAL PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  cognome VARCHAR(50) NOT NULL,
  username VARCHAR(50) NOT NULL,
  password_hash VARCHAR(60) NOT NULL,
  data_nascita DATE,
  luogo_domicilio VARCHAR(50) NOT NULL,
  is_ristoratore BOOLEAN NOT NULL );
```

RistorantiTheKnife

Contiene i dati anagrafici e commerciali del ristorante, incluse le coordinate geografiche (latitude/longitude, usate per il calcolo delle distanze) e il riferimento al gestore proprietario (`utente_id`, FK verso utenti). Il campo `price` memorizza la fascia di prezzo come sequenza ripetuta del simbolo di valuta (es. «€€€»); il campo numerico `prezzo_medio` è mantenuto come valore coerente con la fascia per abilitare filtri di ricerca quantitativi.

Qui le query SQL usate per la creazione della tabella nel database:

```
CREATE TABLE ristorantitheknife (
  id_ristorante SERIAL PRIMARY KEY,
  "Name" VARCHAR(200), address VARCHAR(256),
  "Location" VARCHAR(300),
  price VARCHAR(300),
  cuisine VARCHAR(300),
  longitude REAL,
  latitude REAL,
  phonenumber VARCHAR(20),
  url VARCHAR(500),
  websiteurl VARCHAR(500),
  award VARCHAR(300),
  greenstar INT,
  facilitiesandservices VARCHAR(500),
  description VARCHAR(2048),
  ordina_online BOOLEAN,
  fa_deliveroo BOOLEAN,
  prezzo_medio NUMERIC(6, 2),
```

```
utente_id INT,
CONSTRAINT fk_ristoranti_utente FOREIGN KEY (utente_id) REFERENCES utenti(id) ON DELETE SET NULL );
```

Recensioni

Associa un utente-cliente a un ristorante con un voto (stelle, 1-5) e un testo libero. Il campo `risposta_gestore` è nullable e ospita l'eventuale replica del gestore proprietario del ristorante: per vincolo applicativo (non di schema) può esistere al più una risposta per recensione.

Qui le query SQL usate per la creazione della tabella nel database:

```
CREATE TABLE recensioni (
  id SERIAL PRIMARY KEY,
  ristorante_id INT NOT NULL,
  utente_id INT NOT NULL,
  stelle INT NOT NULL,
  testo TEXT,
  data_recensione TIMESTAMP DEFAULT NOW(),
  risposta_gestore TEXT,
  CONSTRAINT fk_recensioni_ristorante FOREIGN KEY (ristorante_id) REFERENCES ristoranti(id) ON
  DELETE CASCADE,
  CONSTRAINT fk_recensioni_utente FOREIGN KEY (utente_id) REFERENCES utenti(id) ON DELETE CASCADE
);
```

Preferiti

Tabella di giunzione che realizza la relazione molti-a-molti tra utenti e ristoranti («wishlist» dei clienti). L'inserimento duplicato è prevenuto da un vincolo di unicità sulla coppia (`utente_id`, `ristorante_id`), la cui violazione viene anch'essa intercettata via `SQLState` in `DatabaseManager.aggiungiPreferito`.

Qui le query SQL usate per la creazione della tabella nel database:

```
CREATE TABLE preferiti (
  utente_id INT NOT NULL,
  ristorante_id INT NOT NULL,
  PRIMARY KEY (utente_id, ristorante_id),
  CONSTRAINT fk_preferiti_utente FOREIGN KEY (utente_id) REFERENCES utenti(id) ON DELETE CASCADE,
  CONSTRAINT fk_preferiti_ristorante FOREIGN KEY (ristorante_id) REFERENCES ristoranti(id) ON
  DELETE CASCADE );
```

4. Strutture dati utilizzate

Struttura	Impiego	Motivazione
Map<String,String> (HashMap)	campo params di Messaggio	rappresentazione generica e type-agnostic dei parametri del protocollo, evita di dover ridefinire un DTO per ogni operazione di richiesta
List<T> (ArrayList, via Gson/Arrays.asList)	collezioni di Ristorante / Recensione scambiate come payload JSON	ordine di iterazione prevedibile, compatibilità diretta con array JSON
Set<Integer> (HashSet)	deduplica id_ristorante già caricati in HomeFrame.caricaMieRecensioni	verifica di appartenenza O(1) per evitare richieste ridondanti al server
DefaultTableModel / JTable	presentazione tabellare di ristoranti e recensioni	componente Swing nativo; la prima colonna nasconde l'oggetto di dominio (Ristorante) associato alla riga, riletto al doppio click
StringBuilder	costruzione dinamica delle query SQL con filtri opzionali (cercaRistoranti) e delle liste di parametri (List<Object> params)	evita concatenazioni ripetute di String immutabili e permette query con numero variabile di clausole WHERE

5. Scelte algoritmiche

5.1 Ricerca geografica per raggio — formula di Haversine

La ricerca dei ristoranti «nelle vicinanze» di un punto (lat, lon) calcola la distanza tra il punto di ricerca e ciascun ristorante direttamente in SQL, tramite la formula di Haversine espressa con le funzioni trigonometriche native di PostgreSQL, sia nella SELECT (per restituire la distanza) sia nella clausola WHERE (per filtrare entro il raggio richiesto):

Il raggio terrestre medio (6371 km) è il coefficiente di scala della formula. Il filtro viene applicato sia come colonna calcolata (per l'ORDER BY per distanza crescente) sia ripetuto nella clausola WHERE (complessità O(n) sul numero di ristoranti, accettabile senza indice spaziale per i volumi di dati attesi). Quando le coordinate non sono disponibili (geocoding fallito o non richiesto), il metodo DatabaseManager.cercaRistoranti ricade su un filtro testuale LIKE sul campo Location.

5.2 Geocoding diretto e inverso

La conversione nome-località → coordinate è delegata al servizio pubblico Nominatim (OpenStreetMap), invocato via HTTP GET con URL-encoding del parametro di ricerca e un header User-Agent identificativo (requisito di utilizzo accettabile del servizio). La risposta (un array JSON, di cui si considera solo il primo risultato tramite limit=1) viene parsata con JsonParser.parseString(...).getAsJsonArray(), senza la definizione di una classe DTO dedicata, data la natura puntuale dell'uso. Il metodo è statico e senza stato, e in caso di qualunque eccezione (rete, parsing, timeout) restituisce null, lasciando al chiamante la decisione di fallback (es. ricerca testuale sulla città).

5.3 Validazione di coerenza tra fascia di prezzo e prezzo medio

Poiché il prezzo medio (valore numerico) e la fascia di prezzo (simbolica, es. «€€») sono raccolti indipendentemente in fase di inserimento/modifica di un ristorante, sia lato client (per un riscontro immediato all'utente) sia lato server (come validazione definitiva, non aggirabile da un client malevolo o modificato) viene applicata la medesima regola di coerenza a fasce:

```
private boolean prezzoCoerente(int livello, double prezzo) {
    return switch (livello) {
        case 1 -> prezzo >= 0 && prezzo < 20;
        case 2 -> prezzo >= 20 && prezzo < 50;
        case 3 -> prezzo >= 50 && prezzo <= 80;
        default -> prezzo >= 80;
    };
}
```

La duplicazione controllata della regola (client per la UX, server per l'integrità) è una scelta architetturale deliberata: il client offre un riscontro immediato senza round-trip di rete, mentre il server resta l'unica fonte di verità e rifiuta comunque richieste incoerenti (IllegalArgumentException sollevata in ClientHandler.prezzoMedioValido).

5.4 Determinazione del simbolo di valuta dalla località

AggiungiRistoranteDialog.simboloValuta(String location) implementa un algoritmo di classificazione basato su corrispondenza di sottostringhe (contains, case-insensitive) tra il testo «città, nazione» e un insieme cablato di alias geografici (città e nomi di nazione in italiano/inglese), per assegnare il simbolo di valuta più plausibile (€, \$, £, ¥, CHF, ...). In assenza di corrispondenza, il fallback è l'euro («€»), scelta coerente con il contesto applicativo primariamente europeo.

5.5 Calcolo di aggregazioni lato SQL

La media dei voti e il numero di recensioni per ristorante non sono denormalizzati in tabella, bensì calcolati a runtime tramite LEFT JOIN tra RistorantiTheKnife e recensioni con COALESCE(AVG(rec.stelle), 0) e COUNT(rec.id), raggruppati per id_ristorante (GROUP BY) ed eventualmente filtrati a valle con HAVING (per il filtro «stelle minime»). Questa scelta evita anomalie di consistenza tra il dato denormalizzato e le recensioni sottostanti, al costo di una JOIN aggiuntiva per ogni interrogazione di ricerca.

6. Analisi degli algoritmi

Questo capitolo completa la documentazione del sistema con un'analisi della complessità computazionale (temporale e spaziale) dei principali metodi implementati, individuati attraverso l'esame diretto del codice sorgente. Per i metodi lato server che eseguono query SQL, la complessità è riportata così come eseguita concettualmente dal motore PostgreSQL (assumendo la presenza di indici sulle colonne di ricerca più frequenti, quali id, username e le chiavi esterne).

Metodo / Algoritmo	Complessità	Descrizione
geocodifica (ClientTK.java)	Temporale: $O(1)$ Spaziale: $O(R)$ — R = dimensione risposta JSON	Codifica un nome di luogo in coordinate lat/lon tramite richiesta HTTP GET a Nominatim (OpenStreetMap), con parsing JSON via Gson.
getPrezzoMedioStimato (Ristorante.java)	Temporale: $O(L)$ — L = lunghezza fasciaPrezzo Spaziale: $O(1)$	Switch sulla lunghezza della stringa fasciaPrezzo per restituire un valore di prezzo stimato.
addParam / getParam / hasParam (Messaggio.java)	Temporale: $O(1)$ ammortizzato Spaziale: $O(1)$ per operazione	Operazioni su HashMap per aggiunta, lettura e verifica dei parametri del messaggio.
simboloValuta (AggiungiRistoranteDialog.java)	Temporale: $O(N \times M) \approx O(M)$, N costante (~60 keyword) Spaziale: $O(1)$	Sequenza lineare di controlli contains() su una stringa location per determinare il simbolo di valuta.
prezzoCoerente (AggiungiRistoranteDialog.java)	Temporale: $O(1)$ Spaziale: $O(1)$	Switch con confronti aritmetici per validare la coerenza tra fascia di prezzo e prezzo medio.
tentaInvio / doInBackground (SwingWorker) (AggiungiRistoranteDialog.java)	Temporale: $O(1)$ lato client (esclusa latenza rete/geocoding) Spaziale: $O(1)$	Raccolta dati dal form, geocoding, costruzione del messaggio e invio al server via socket.
caricaDati / popola (DettaglioDialog.java)	Temporale: $O(R)$ — R = numero recensioni Spaziale: $O(R)$	Richiesta al server delle recensioni di un ristorante, seguita da rendering lineare della lista.
estrazione fascia prezzo (loop su fasciaOriginale) (DettaglioDialog.creaSezioneInfo)	Temporale: $O(L)$ — L = lunghezza fasciaPrezzo Spaziale: $O(L)$ per lo StringBuilder	Scorre carattere per carattere la stringa fasciaPrezzo per estrarre simbolo valuta e livello di prezzo.
aggiornaVoceAttiva (HomeFrame.java)	Temporale: $O(V)$ — V = voci menu, costante (~6) Spaziale: $O(1)$	Scorre la lista vociMenu e aggiorna font/colore di ciascuna voce per evidenziare quella attiva.
cercaRapida / cercaAvanzata / eseguiCerca (HomeFrame.java)	Temporale: $O(N)$ — N = ristoranti restituiti Spaziale: $O(N)$	Geocoding, costruzione del messaggio con filtri, invio al server, deserializzazione JSON e popolamento tabella.
popolaTabella (HomeFrame.java)	Temporale: $O(N)$ — N = numero ristoranti Spaziale: $O(N)$	Itera sulla lista di ristoranti e inserisce ogni riga nel DefaultTableModel.
caricaMieRecensioni (HomeFrame.java)	Temporale: $O(R + U)$ — R = recensioni, U = ristoranti distinti Spaziale: $O(R + U)$	Recupera tutte le recensioni dell'utente e, per ogni ristorante distinto, effettua una richiesta separata per il nome; usa un HashSet per la deduplica.

Metodo / Algoritmo	Complessità	Descrizione
nomeRistoranteMieRecensione (HomeFrame.java)	Temporale: $O(U)$ — U = ristoranti distinti caricati Spaziale: $O(1)$	Ricerca lineare nella lista ristorantiMieRecensioni per trovare il nome dato l'ID.
caricaRecensioniGestore (HomeFrame.java)	Temporale: $O(K \times T)$ rete + $O(R)$ rendering Spaziale: $O(R + K)$	Recupera i ristoranti del gestore, poi per ognuno effettua una richiesta separata al server per le recensioni.
nomeRistoranteGestore (HomeFrame.java)	Temporale: $O(K)$ — K = ristoranti del gestore Spaziale: $O(1)$	Ricerca lineare nella lista ristorantiGestorePerRecensioni.
popola / calcolo media stelle (RecensioniDialog.java)	Temporale: $O(R)$ — R = numero recensioni Spaziale: $O(R)$	Usa uno Stream Java per calcolare la media delle stelle, poi rendering lineare delle card recensioni.
getInstance (Singleton) (SessioneUtente.java, DatabaseManager.java)	Temporale: $O(1)$ Spaziale: $O(1)$	Pattern Singleton con controllo null e synchronized: crea l'istanza solo al primo accesso.
elabora / switch operazioni (ClientHandler.java)	Temporale: $O(1)$ — numero di casi fisso Spaziale: $O(1)$	Switch sulla stringa operazione per smistare il messaggio verso il metodo handler corretto.
prezzoMedioValido (ClientHandler.java)	Temporale: $O(1)$ Spaziale: $O(1)$	Switch con confronti numerici per validare, lato server, la coerenza tra prezzo e fascia.
login (DatabaseManager.java)	Temporale: $O(B)$ BCrypt (costante, 2^{12}) + $O(\log N)$ query indicizzata Spaziale: $O(1)$	Query SQL SELECT per username, seguita da verifica BCrypt della password.
registraUtente (DatabaseManager.java)	Temporale: $O(B)$ BCrypt + $O(\log N)$ INSERT indicizzato Spaziale: $O(1)$	Hash BCrypt della password, INSERT SQL con RETURNING id; gestisce l'eccezione per vincolo UNIQUE.
modificaUtente (DatabaseManager.java)	Temporale: $O(B)$ se include hashing, altrimenti $O(\log N)$ Spaziale: $O(1)$	UPDATE SQL condizionale (con o senza password), con costruzione dinamica della query.
cercaRistoranti (DatabaseManager.java)	Temporale: $O(N \times F)$ nel DBMS — N = ristoranti, F = filtri applicati Spaziale: $O(N)$ per la lista risultati	Algoritmo più complesso del progetto: costruisce dinamicamente una query SQL con filtri opzionali (cucina, prezzo, delivery, prenotazione, stelle minime) e calcola la distanza reale con la formula di Haversine direttamente in SQL.
ristorantePerId (DatabaseManager.java)	Temporale: $O(R)$ per media/conteggio, $O(\log N)$ per l'accesso Spaziale: $O(1)$	SELECT con JOIN e GROUP BY su un singolo id, calcola media voto e numero recensioni.
aggiungiRistorante (DatabaseManager.java)	Temporale: $O(\log N)$ con indice Spaziale: $O(1)$	INSERT SQL con 15 parametri e RETURNING id.
ristorantiDelGestore (DatabaseManager.java)	Temporale: $O(K \times R)$ nel caso peggiore, ordinamento $O(K \log K)$ Spaziale: $O(K)$	SELECT con LEFT JOIN recensioni, GROUP BY e ORDER BY alfabetico sul nome.

Metodo / Algoritmo	Complessità	Descrizione
aggiungiRecensione (DatabaseManager.java)	Temporale: $O(\log N)$ con indice Spaziale: $O(1)$	Validazione delle stelle $O(1)$ seguita da INSERT SQL.
modificaRecensione (DatabaseManager.java)	Temporale: $O(\log N)$ check + $O(P)$, P = campi modificati (max 2) Spaziale: $O(P)$	SELECT preliminare per verificare l'appartenenza, poi UPDATE con lista parametri costruita dinamicamente.
eliminaRecensione (DatabaseManager.java)	Temporale: $O(\log N)$ con indice Spaziale: $O(1)$	DELETE SQL con WHERE su id e utente_id.
rispondiRecensione (DatabaseManager.java)	Temporale: $O(\log N)$ per entrambe le query, con indici Spaziale: $O(1)$	SELECT con JOIN per verificare la proprietà del ristorante, poi UPDATE con subquery EXISTS.
recensioniRistorante / recensioniUtente (DatabaseManager.java)	Temporale: $O(R \log R) - R$ = numero recensioni Spaziale: $O(R)$	SELECT con JOIN utenti, filtro su ristorante_id o utente_id, ORDER BY data decrescente.
aggiungiPreferito / rimuoviPreferito (DatabaseManager.java)	Temporale: $O(\log N)$ con indice Spaziale: $O(1)$	INSERT / DELETE sulla tabella preferiti, con gestione dell'eccezione UNIQUE per il caso duplicato.
preferiti (DatabaseManager.java)	Temporale: $O(F \times R)$ nel DBMS, ordinamento $O(F \log F)$ Spaziale: $O(F)$	SELECT con INNER JOIN preferiti e LEFT JOIN recensioni, GROUP BY e ORDER BY alfabetico.
eseguiQueryRecensioni (DatabaseManager.java)	Temporale: $O(R)$ — iterazione sul ResultSet Spaziale: $O(R)$	Metodo helper riusato da recensioniRistorante e recensioniUtente: esegue la query e costruisce la lista di Recensione.
normalizzaPrezziMedi (DatabaseManager.java)	Temporale: $O(N)$ — scansione intera tabella Spaziale: $O(1)$, eseguito nel DBMS	UPDATE bulk con CASE/WHEN sui ristoranti con prezzo incoerente; usa FLOOR(random()) per generare valori nell'intervallo corretto.
hash / verifica (PasswordUtil.java)	Temporale: $O(2^W)$, W = work factor (12) → $O(4096)$, costante Spaziale: $O(1)$	Hashing BCrypt (work factor 12) e verifica a tempo costante, immune a timing attack.
avviaAccettazione (accept loop) (ServerGUI.java)	Temporale: $O(1)$ per accettazione, loop $O(\infty)$ finché attivo Spaziale: $O(C)$, C = client connessi	Thread daemon che accetta connessioni in loop; per ogni client lancia un nuovo Thread con ClientHandler.
log / rendering log colorato (ServerGUI.java)	Temporale: $O(L)$ — L = lunghezza del messaggio Spaziale: $O(L \times T)$, T = messaggi totali loggati	Inserisce testo stilizzato nello StyledDocument del JTextPane con timestamp, tag e messaggio.
creaTabella / renderer celle (UI.java)	Temporale: $O(1)$ configurazione, $O(1)$ per cella Spaziale: $O(1)$	Configura la JTable con renderer custom per colori alternati (zebra striping) ed evidenziazione della selezione.

7. Pattern applicati

I pattern nel codice si classificano in due livelli: i pattern architetturali, che riguardano l'organizzazione complessiva del sistema e la distribuzione dei processi/thread, e i pattern di progettazione, relativi alla struttura interna delle singole classi e alle loro collaborazioni. Per ciascun pattern si indicano le classi coinvolte, la motivazione della scelta e un estratto di codice rappresentativo.

7.1 Pattern architetturali

7.1.1 Client-Server

È il pattern architetturale primario del sistema: due processi distinti e distribuibili su host diversi (client Swing e server) comunicano tramite un protocollo di rete (socket TCP), con il server che espone servizi (autenticazione, ricerca, gestione recensioni) e il client che li consuma. Il server è «stateful per connessione» (mantiene l'utente autenticato nell'istanza di `ClientHandler` associata al socket) ma stateless tra connessioni diverse, e costituisce l'unico punto di accesso alla base dati: nessun client dialoga direttamente con PostgreSQL.

7.1.2 Master/Worker (Master/Slave)

Il server applica il classico pattern concorrente Master/Worker (talvolta indicato come Master/Slave) per la gestione delle connessioni in ingresso: un unico thread «master» (`ServerGUI.acceptThread`) esegue un ciclo bloccante su `ServerSocket.accept()` e, per ogni connessione accettata, crea e avvia un nuovo thread «worker/slave» dedicato (un'istanza di `ClientHandler` eseguita come thread daemon), a cui delega per intero la gestione del ciclo di vita di quel client. Il master non partecipa all'elaborazione delle richieste: il suo unico compito è l'accettazione e la distribuzione del lavoro. I worker condividono una risorsa comune (il Singleton `DatabaseManager`), il cui accesso è serializzato tramite `synchronized` per evitare corruzione dello stato condiviso (connessione JDBC).

```
// ServerGUI.avviaAccettazione(porta) – thread MASTER
acceptThread = new Thread(() -> {
    while (running) {
        Socket client = serverSocket.accept();    // punto di sincronizzazione del master
        Thread t = new Thread(new ClientHandler(client, ip, logger)); // spawna un WORKER
        t.setDaemon(true);
        t.start();                                // il master torna subito in ascolto
    }
});
```

A differenza di un pool di thread dimensionato staticamente (Thread Pool pattern), qui il numero di worker è dinamico e pari al numero di client connessi (un thread per connessione, non per richiesta): scelta semplice e adeguata al carico atteso, ma che non limita esplicitamente il numero massimo di thread concorrenti — un'estensione naturale sarebbe l'introduzione di un `ExecutorService` con pool dimensionato.

7.1.3 Architettura orientata ai messaggi (Message-oriented / Protocollo a operazioni)

Client e server non espongono API a livello di trasporto (non vi sono, ad esempio, endpoint REST o chiamate RPC dirette sui metodi Java): ogni interazione è mediata da un unico «envelope» generico (Messaggio) scambiato come singolo messaggio JSON di linea. Questo è un pattern architetturale di integrazione a messaggi (message-oriented, in forma sincrona request/response su un singolo canale), che disaccoppia la forma del payload dalla struttura del trasporto e rende il protocollo facilmente estendibile con nuove operazioni senza modificare il framing a livello di socket.

7.2 Pattern di progettazione (GoF ed enterprise)

7.2.1 Singleton

Applicato a tre classi con responsabilità di «stato/risorsa unica per processo»: `SessioneUtente` (stato di sessione lato client), `ServerGUI` (accesso statico all'istanza per il logging da `ClientHandler`). In tutti i casi l'accesso avviene tramite un metodo statico `getInstance()` con inizializzazione lazy; per `SessioneUtente` il metodo è `synchronized` per garantire la corretta pubblicazione dell'istanza in presenza di più thread (double-checked locking non necessario data la bassa frequenza di primo accesso).

```
public static synchronized DatabaseManager getInstance() {
    if (instance == null) instance = new DatabaseManager();
    return instance;
}
```

7.2.2 Data Access Object (DAO)

`DatabaseManager` svolge il ruolo di DAO verso l'intero schema applicativo: nessun'altra classe del server costruisce direttamente istruzioni SQL. `ClientHandler` dipende esclusivamente dall'interfaccia (i metodi pubblici) di `DatabaseManager`, ignorando lo schema fisico delle tabelle. È un pattern enterprise (Core J2EE Patterns) piuttosto che GoF, ma è tra i più determinanti nella struttura del server.

7.2.4 Observer / Callback

L'interfaccia `ClientHandler.ServerLogger` disaccoppia il thread di gestione della connessione dagli aggiornamenti della GUI del server, realizzando un canale di notifica asincrona (log operazione, log errore, disconnessione client) verso un osservatore — nella pratica, sempre l'unica istanza di `ServerGUI`, ma l'interfaccia consente in linea di principio implementazioni alternative (es. logging su file per una modalità headless, come suggerito dalla presenza residua di `ServerTK`).

7.2.5 Template Method

Ogni utilizzo di `javax.swing.SwingWorker<T,Void>` nel client (decine di occorrenze in `HomeFrame`, `LoginFrame`, `DettaglioDialog`, ...) è un'istanza del pattern Template Method: la classe astratta `SwingWorker` definisce lo scheletro invariante dell'algoritmo (esecuzione su thread separato, poi pubblicazione del risultato sull'EDT), mentre ogni sottoclasse anonima ne specializza i due passi variabili `doInBackground()` (il «cosa», tipicamente una chiamata `ClientTK.invia()`) e `done()` (l'aggiornamento della GUI). Il chiamante non invoca mai questi metodi direttamente: è il framework Swing a gestirli tramite `execute()`.

```
new SwingWorker<Boolean, Void>() {
    @Override protected Boolean doInBackground() throws Exception { /* passo variabile 1 */ }
    @Override protected void done() { /* passo variabile 2 */ }
}.execute(); // scheletro fisso, definito da SwingWorker
```

7.2.6 Factory (statica) / Utility

La classe `theknife.gui.UI` centralizza, con soli metodi statici, la creazione di componenti Swing preconfigurati secondo il tema grafico dell'applicazione (`creaInput`, `creaBottone`, `creaTabella`, `creaCardPanel`, ...). Pur non essendo un Factory Method in senso stretto (non vi è polimorfismo né interfaccia di prodotto), la classe assolve la medesima funzione di disaccoppiare la costruzione dei componenti con tutti i dettagli di styling dal codice dei singoli frame/dialog, eliminando la duplicazione di codice di styling che altrimenti ricorrerebbe in ogni classe GUI.

7.2.7 Data Transfer Object (DTO) / Value Object

Le classi di theknife.common (Utente, Ristorante, Recensione, Messaggio) sono Data Transfer Object: attraversano il confine di processo client-server serializzate in JSON, sono prive di dipendenze da Swing o da JDBC, e la loro unica logica è la validazione/derivazione di valori di comodo.

7.2.8 Facade

ClientTK agisce da Facade verso il sottosistema di comunicazione: le classi GUI non manipolano mai direttamente Socket, stream o Gson, ma invocano la semplice API connetti/invia/disconnetti. In modo simmetrico, DatabaseManager è una Facade verso JDBC per l'intero server: ClientHandler non vede mai un oggetto Connection, PreparedStatement o ResultSet.

Limiti della soluzione sviluppata

Scalabilità e prestazioni

- La ricerca dei ristoranti calcola la formula di Haversine riga per riga in SQL, senza indice spaziale (es. PostGIS/GiST): il costo cresce linearmente con il numero di ristoranti e manca la paginazione dei risultati.
- DatabaseManager è un singleton con un'unica Connection JDBC condivisa e metodi synchronized: l'accesso al database è di fatto serializzato, senza connection pool.
- Il server crea un nuovo Thread per ogni client accettato, senza un pool dimensionato: il numero di thread cresce senza limiti con le connessioni simultanee.

Persistenza e affidabilità

- Nessuna riconnessione automatica al database: se la connessione JDBC cade, il server richiede un riavvio manuale.
- Operazioni composte da più passi (es. rispondiRecensione, modificaRecensione) non sono racchiuse in transazioni esplicite, con possibili race condition check-then-act in caso di richieste concorrenti.
- Assenza di backup/ripristino automatico e di strumenti di versionamento dello schema (es. Flyway, Liquibase).

Sicurezza

- Comunicazione client/server su socket TCP privo di TLS: password e dati applicativi transitano in chiaro sulla rete (la password resta comunque protetta a riposo tramite BCrypt).
- La sessione resta valida per l'intera durata del socket, senza timeout né possibilità di invalidazione remota.

Qualità e completezza dei dati

- La geocodifica dipende dal servizio gratuito Nominatim, senza cache dei risultati e soggetto a rate limiting.
- Diverse validazioni esistono solo lato client (form Swing) e non sono ripetute lato server, che quindi non intercetta dati incompleti o incoerenti inviati direttamente via protocollo.
- Assenza del caricamento immagini; nessuna moderazione automatica dei testi delle recensioni.

Internazionalizzazione e interfaccia

- Interfaccia disponibile solo in italiano, con stringhe hardcoded nel codice.
- Tema grafico scuro fisso, non personalizzabile dall'utente.
- Applicazione desktop Swing, senza versione web o mobile, e componenti grafici custom non pensati per le tecnologie assistive.

Sitografia / Bibliografia

Documentazione tecnica

- Oracle Java Documentation, Online: <https://docs.oracle.com/en/java/>
- Oracle, "Swing Tutorial — Creating a GUI With JFC/Swing", Online: <https://docs.oracle.com/javase/tutorial/uiswing/>
- Gson Library Documentation (Google), Online: <https://github.com/google/gson>
- PostgreSQL JDBC Driver Documentation, Online: <https://jdbc.postgresql.org/documentation/>
- PostgreSQL Official Documentation, Online: <https://www.postgresql.org/docs/>
- java-bcrypt Library Documentation (P. Favre-Bulle), Online: <https://github.com/patrickfav/bcrypt>
- Nominatim API Documentation (OpenStreetMap Foundation), Online: <https://nominatim.org/release-docs/latest/api/Overview/>
- OpenStreetMap Nominatim Usage Policy, Online: <https://operations.osmfoundation.org/policies/nominatim/>

Guide di riferimento

- "Java: The Complete Reference" di Herbert Schildt, McGraw-Hill Education, 2020
- "Effective Java" di Joshua Bloch, Addison-Wesley Professional, 2018
- "Java Concurrency in Practice" di Brian Goetz et al., Addison-Wesley, 2006
- "PostgreSQL: Up and Running" di Regina Obe e Leo Hsu, O'Reilly Media, 2017

Standard e convenzioni

- RFC 8259 — The JavaScript Object Notation (JSON) Data Interchange Format, Online: <https://www.rfc-editor.org/rfc/rfc8259>
- ISO 8601 Data and Time Format, Online: <https://www.iso.org/iso-8601-date-and-time-format.html>
- Niels Provos, David Mazières, "A Future-Adaptable Password Scheme" (algoritmo bcrypt), USENIX Annual Technical Conference, 1999, Online: <https://www.usenix.org/legacy/events/usenix99/provos.html>
- "Calculate distance, bearing and more between Latitude/Longitude points" (formula di Haversine), Movable Type Scripts, Online: <https://www.movable-type.co.uk/scripts/latlong.html>